INTERNSHIP PROJECT · TÜRKSAT AVIATION AND COMMUNICATION

Smart School Information System

Week 2 — FastAPI Skeleton and Authentication

PREPARED BY	Bera Ergin — s29315
INSTITUTION	PJAIT — 7th Semester

THIS WEEK	Week 1's RLS guarantee is carried through every request in the API

Where We're Coming From, Where We're Going

WEEK 1 — DONE

- ✓ PostgreSQL + Row-Level Security set up
- ✓ 3 tests: teacher→4, principal→12, parent→1
- ✓ No API, no user login at all
- ✓ Only the database, correctly filtering on its own

No one can log in yet, and nothing sets the RLS context.

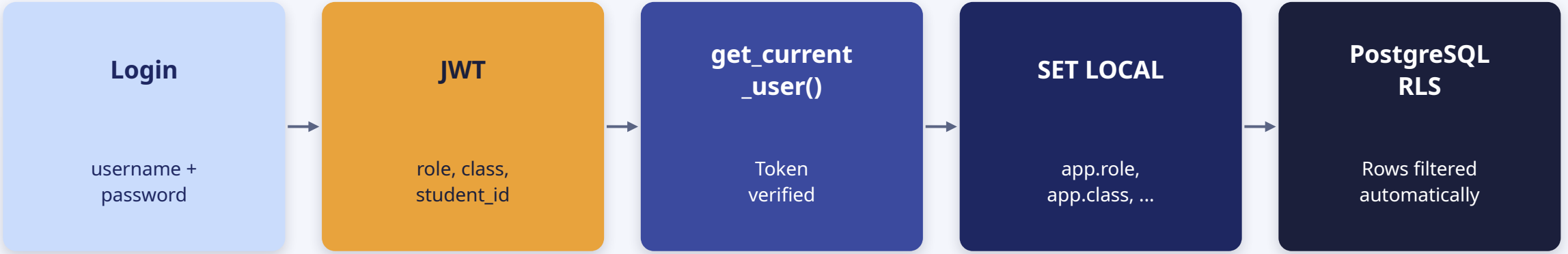
WEEK 2 — GOAL

- FastAPI + JWT login
- The logged-in user's role/class is determined
- This is declared to the PostgreSQL session on every request
- Week 1's RLS proof is now verified through the API too

No LLM/query pipeline yet — just identity and the context chain.

Context Chain: Login to RLS

The chain that lets Week 1's RLS kick in automatically on every API request



Critical design decision

FastAPI never decides "this user can see these rows" — it only verifies identity and declares that identity to the PostgreSQL session. Which rows come back is decided by the RLS policies set up in Week 1. The API layer never knows the access rule itself.

Authentication Flow

Username/password → JWT token → protected endpoints



Example JWT payload

```
{ "sub": "teacher1", "role": "teacher", "related_class": "9-A", "exp": 1735729200 }
```

Password Security

Moving from Week 1's plaintext passwords to bcrypt hashes

Week 1

```
password_hash = "teacher123"
```

Stored as plaintext — documented in WEEK_01.md as a deliberate simplification.

Week 2

```
password_hash = "$2b$12$KIXx..."
```

Generated with get_password_hash() using bcrypt. Column name stays the same — only the content changes, no model change needed.

Why it matters

Even if the database is compromised, passwords can't be read directly. passlib[bcrypt] generates a unique salt per password, so even two users with the same password get different hashes.

Role-Based Login + RLS Verification

Every role logs in and hits the same /debug/students endpoint — the returned row count must match Week 1's results

Role	Username	Context	Expected Result
Principal	`principal`	—	200 + 12 rows
Teacher	`teacher1`	9-A	200 + 4 rows
Parent	`parent1`	Ali Yılmaz	200 + 1 row
Student	`student1`	Ali Yılmaz	200 + 1 row
Invalid	`teacher1 / wrong password`	—	**401 Unauthorized**

This table shows Week 1's test_rls.py results (4/12/1) now verified identically through the API layer, with a real user login.

A Temporary Proof Endpoint

GET /debug/students — deliberately temporary, deliberately documented

There's no LLM/query pipeline yet this week — but we need some way to prove the context chain (login → JWT → SET LOCAL → RLS) actually works. A simple, fixed endpoint that only runs "SELECT id, name, class_name FROM students" was added for exactly that.

This endpoint lives in its own file (backend/app/api/debug.py) and will be removed once the LLM integration arrives in Week 4 — the real /query endpoint will take its place.

Why its own file

It's not mixed into query.py — so removing this temporary code in Week 4 is as simple as deleting one file, without touching the real query logic at all.

Week 2 — Scope and Acceptance Criteria

Delivered this week

- ✓ FastAPI project skeleton (main.py, routers)
- ✓ security.py — JWT + bcrypt helpers
- ✓ auth.py — POST /auth/login endpoint
- ✓ dependencies.py — get_current_user() + RLS context
- ✓ A separate school_app connection with SET LOCAL
- ✓ Temporary /debug/students — API-level proof of RLS

Acceptance criteria

1. uvicorn backend.app.main:app runs, /docs is reachable
2. Each of the 4 roles returns 200 + token with the correct password
3. A wrong password returns 401 Unauthorized
4. /debug/students matches Week 1's counts (4/12/1/1) for each role
5. The school_app role is not a superuser — RLS is genuinely active

Next Step — Week 3

Scope under re-evaluation

The original plan had Week 3 building a RuleEngine class — Python code generating role-based SQL filters. But PostgreSQL itself now does that job, at the database level, via the RLS set up in Weeks 1-2. Week 3's scope will be redefined to reflect this.

Thank you